

JAVA 17 OCP · YDS STUDY SERIES

Unit 08 Grammar Notes · Lambdas and Functional Interfaces

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya, [çift dilli Chapter 8 notundaki](#) gerçek cümle yapılarını Java/YDS bağlamında açıklar. Teknik kelimeler için [Unit 08 vocabulary](#) kaynağına bakın.

1. allow + object + to-infinitive

EN: Lambdas allow you to specify code that will be run later in the program.

TR: Lambda'lar programın ilerleyen bölümünde çalıştırılacak kodu belirtmenize olanak tanır.

Formül:

```
allow + object + to + base verb
```

`Allow you specify` active yapıda yanlıştır; object'ten sonra `to` gerekir. Passive biçimde `be allowed to + verb` kullanılır: 'You are allowed to omit the parentheses.'

YDS ipucu: `enable + object + to-infinitive` aynı pattern'ı izler; `let + object + bare infinitive` ise `to` almaz.

2. a way of + gerund

EN: Functional programming is a way of writing code more declaratively.

TR: Functional programming, kodu daha declarative biçimde yazmanın bir yoludur.

Formül:

```
a way of + V-ing
```

Preposition `of` sonrasında base verb değil gerund gelir. Alternatif yapı `a way to write code` biçimidir.

Sık hata: `a way of write` değil, `a way of writing`.

3. rather than + parallel structure

EN: You specify what you want to do rather than dealing with the state of objects.

TR: Object'lerin state'iyile uğraşmak yerine ne yapmak istediğinizi belirtirsiniz.

`rather than` iki seçeneği karşılaştırır. İki tarafın grammatical form'u mümkün olduğunca paralel olmalıdır:

```
verb phrase + rather than + V-ing / verb phrase
```

Buradaki doğal Türkçe karşılık "... yerine"dir.

4. according to + noun phrase

EN: Our goal is to print out all the animals in a list according to some criteria.

TR: Amacımız bir listedeki bütün hayvanları bazı ölçütlere göre yazdırmaktır.

`according to`, "-e göre / uyarınca" anlamında preposition'dır ve ardından noun phrase gelir.

YDS ipucu: *according to me* gündelik kullanımda doğal değildir; kişisel görüş için *in my opinion*, kaynak/kurala dayalı bilgi için *according to* kullanılır.

5. Indirect question: *whether + clause*

EN: The first thing we want to check is whether the *Animal* can hop.

TR: Kontrol etmek istediğimiz ilk şey *Animal* nesnesinin zıplayıp zıplayamadığıdır.

Whether, yes/no indirect question kurar:

```
whether + subject + verb
```

Indirect question'da soru word order'ı kullanılmaz: *whether can the Animal hop* değil, *whether the Animal can hop*.

6. Concession: *granted, ... but ...*

EN: Granted, it is only a few lines, but it is a whole new file.

TR: Yalnız birkaç satırdan oluştuğu doğru olsa da tamamen yeni bir dosyadır.

Granted karşı tarafın bir noktasını kabul edip ardından contrast kurar.

```
Granted, concession, but main argument.
```

YDS'de *admittedly, although it is true that* ile yakın işlevdedir.

7. *no need for + noun + to-infinitive*

EN: We only have to add one line of code—no need for an extra class to do something simple.

TR: Yalnızca bir satır kod eklememiz gerekir; basit bir şey yapmak için fazladan class'a ihtiyaç yoktur.

Formül:

```
there is no need for + noun  
there is no need for + noun + to + verb
```

No need of bu bağlamda doğal tercih değildir.

8. *as opposed to*

EN: In this case, “later” is inside the *print()* method body, as opposed to when it is passed to the method.

TR: Bu durumda “daha sonra”, kodun method'a geçirildiği anın aksine *print()* method body'sinin içidir.

as opposed to, iki öge arasında açık contrast kurar ve “-nın aksine / karşısında” anlamına gelir. *to* preposition olduğu için ardından noun, pronoun veya gerund gelebilir.

9. Causal *since*

EN: Since that interface's method takes an *Animal*, the lambda parameter has to be an *Animal*.

TR: Interface'in method'u bir *Animal* aldığı için lambda parameter'ı da bir *Animal* olmak zorundadır.

Buradaki *since* zaman değil reason bildirir:

```
Since + reason clause, result clause.
```

YDS ipucu: Bir başlangıç zamanı/perfect tense yoksa ve açık sebep–sonuç ilişkisi varsa “çünkü / -dığı için” anlamını dene.

10. *only if* as a necessary condition

EN: The parentheses around the lambda parameters can be omitted only if there is a single parameter and its type is not explicitly stated.

TR: Lambda parameter'larını çevreleyen parentheses yalnızca tek bir parameter varsa ve bu parameter'ın type'ı explicit olarak belirtilmemişse atlanabilir.

only if, necessary condition (gerekli koşul) bildirir:

```
result only if necessary condition
```

A *only if* B, A'nın gerçekleşmesi için B'nin gerekli olduğunu söyler. *if* ile yer değiştirildiğinde logic değişebilir.

11. *even though* concession

EN: Even though default methods function like abstract methods, in that they can be overridden in a class implementing the interface, they are insufficient for satisfying the single abstract method requirement.

TR: Default method'lar, interface'i implement eden bir class'ta override edilebilmeleri bakımından abstract method'lar gibi işlev görse de single abstract method gereksinimini karşılamak için yetersizdir.

Formül:

```
even though + full clause, contrasting result
```

Despite sonrasında doğrudan full clause gelmez; `despite the fact that + clause *veya* despite + noun/gerund` gerekir.

12. *just because ... does not mean ...*

EN: However, just because you don't see the annotation doesn't mean it's not a functional interface.

TR: Ancak annotation'ı görmemeniz, onun functional interface olmadığı anlamına gelmez.

Bu kalıp hatalı bir çıkarımı reddeder:

```
Just because + clause A + does not mean + clause B
```

Türkçeye çoğu zaman “Yalnızca A olması, B olduğu anlamına gelmez” biçiminde çevrilir.

13. **while** for contrast

EN: While all method references can be turned into lambdas, the opposite is not always true.

TR: Bütün method reference'lar lambda'ya dönüştürülebilse de bunun tersi her zaman doğru değildir.

Buradaki **while** eşzamanlılık değil concession/contrast bildirir. Clause'lar arasında “-se de / oysa” anlamı kurar.

YDS ipucu: **while** clause'larında iki zıt genel gerçek varsa contrast; eşzamanlı iki eylem varsa “-iken” anlamını düşün.

14. **by + gerund**: yöntem

EN: We implement it by checking if the **String** is empty:

TR: Bunu **String**in empty olup olmadığını kontrol ederek implement ediyoruz.

by + V-ing, bir sonucun hangi yöntemle elde edildiğini açıklar:

```
do something + by + V-ing
```

By **check** değil, **by checking** kullanılır.

15. **without + gerund**

EN: A **Supplier** is used when you want to generate or supply values without taking any input.

TR: Herhangi bir input almadan değer üretmek veya sağlamak istediğinizde **Supplier** kullanılır.

Preposition **without** sonrasında gerund gelir:

```
without + V-ing
```

Gizli subject ana clause'un subject'iyle aynı kabul edilir. Subject farklıysa **without + noun/pronoun + V-ing** yapısı kurulabilir.

16. **the + comparative, the + comparative**

EN: The shorter the lambda, the easier it is to read the code.

TR: Lambda ne kadar kısa olursa kodu okumak o kadar kolay olur.

Correlative comparative yapısı iki değişimin birlikte ilerlediğini gösterir:

```
the + comparative + clause, the + comparative + clause
```

Türkçede “ne kadar ... o kadar ...” veya “...dıkça ...” ile çevrilir.

17. Reduced clause: when + past participle

EN: Which lambda expression, when entered into the blank line in the following code, causes the program to print `hahaha`?

TR: Aşağıdaki koddaki boş satıra girildiğinde hangi lambda expression programın `hahaha` yazdırmasını sağlar?

Tam passive clause şudur:

```
when it is entered into the blank line
```

Subject ve `be` düşürülerek `when entered ...` biçimi oluşur. Gizli subject, modified noun olan `lambda expression`dir.

YDS ipucu: `when/if/although + V3` çoğunlukla reduced passive clause'dur.

18. as long as: koşul

EN: A lambda can define parameters or variables in the body as long as their names are different from existing local variables.

TR: Lambda, adları mevcut local variable'lardan farklı olduğu sürece body'de parameter veya variable tanımlayabilir.

`as long as`, bu bağlamda süre değil “şu koşulla / -dığı sürece” anlamı taşır:

```
result + as long as + condition
```

`provided that` ve `so long as` ile yakın anlamlıdır.

19. Hypothetical condition: if + past, would + verb

EN: What would happen if we tried to print out `s3` itself?

TR: `s3`ün kendisini yazdırmaya çalışsaydık ne olurdu?

Bu second conditional pattern varsayımsal sonucu sorar:

```
What would happen if + past simple?
```

Past form burada geçmiş zamanı değil düşük gerçeklik/varsayım anlamını işaretleyebilir.

20. vice versa

EN: Be able to convert method references into regular lambda expressions and vice versa.

TR: Method reference'ları normal lambda expression'lara ve tersini de method reference'lara dönüştürebilir.

`vice versa`, hemen önceki ilişkinin ters yönünün de geçerli olduğunu söyler. İki uzun clause'u tekrar etmeyi önleyen bağlayıcı bir phrase'dir.

Mini quiz · Özgün YDS/teknik İngilizce çalışması

- 1 A lambda may capture a local variable ____ it is effectively final. Boşluğa gerekli koşulu bildiren hangi yapı gelir?
- 2 The method reference is shorter ____ the equivalent lambda. cümlesinde karşılaştırma için hangi sözcük gerekir?
- 3 We resolve the overload ____ examining the target type. cümlesini yöntem bildirecek şekilde tamamlayın.
- 4 Functional programming is a way of ____ (write) code declaratively.
- 5 ____ all method references have lambda equivalents, not all lambdas have method-reference equivalents. Contrast bağlacını yazın.
- 6 The ____ the lambda, the easier the code is to read. Comparative biçimi yazın.

Cevaplar ve kısa açıklamalar

- 1 only if — effectively final olmak gerekli koşuldur.
- 2 than — comparative shorter than.
- 3 by — by examining, yöntemi bildirir.
- 4 writing — of preposition'ından sonra gerund.
- 5 While — burada zaman değil contrast bildirir.
- 6 shorter — correlative comparative: the shorter ..., the easier

Hızlı YDS özeti

```
allow + object + to V
a way of + V-ing
rather than + parallel form
only if + necessary condition
by / without + V-ing
even though / while + clause
the more/shorter ..., the more/easier ...
when + V3 → reduced passive clause
as long as + condition
```